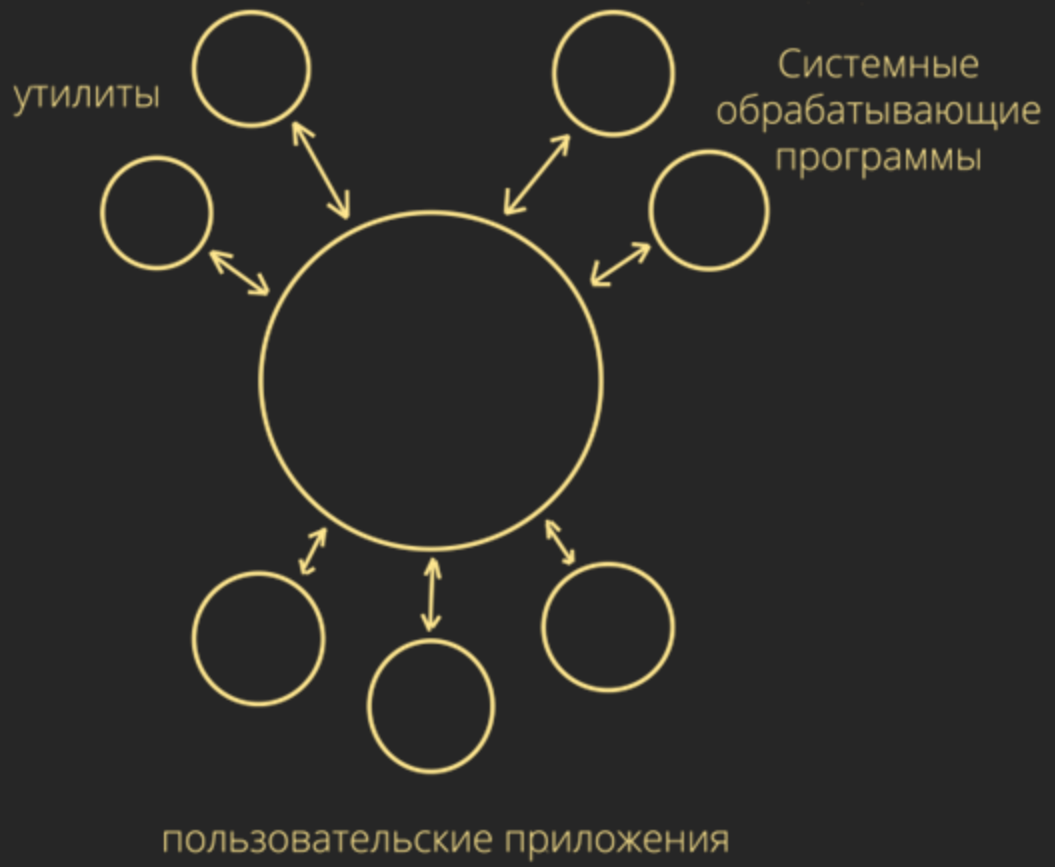
АРХИТЕКТУРА ОПЕРАЦИОННЫХ СИСТЕМ

Архитектура ОС





Дистрибутивы

Дистрибутив – форма распространения системного программного обеспечения.

Дистрибутив состоит из:

- ядра
- системы инициализации
- предустановленного и доступного набора ПО
- графической оболочки по умолчанию
- пакетного менеджера
- исправлений и другого ПО от разработчиков дистрибутива.

программы для начальной инициализации системы:
инициализация аппаратной части, загрузка урезанной версии системы и запуск программы-установщика
программа-установщик
для выбора режимов и параметров установки
набор специальных файлов (пакетов)
содержит отдельные части системы (ядро, прикладные приложения и т.д.).
программа установки позволяет установить и произвести первичную настройку системы.

Ядро выполняет:

Базовые функции ОС:

управление процессами, памятью, устройствами ввода/вывода

Функции, решающие внутрисистемные задачи организации вычислительного процесса:

переключение контекстов, загрузка/выгрузка страниц, обработка прерываний

Функции, создающие прикладную программную среду: API

Компоненты ядра Linux

- Process Scheduler (SCHED) — планировщик процессов, отвечает за контроль над доступом процессов к CPU
- Memory Manager (MM) — менеджер памяти, обеспечивает различным процессам безопасный доступ к основной памяти системы
- Virtual File System (VFS) — виртуальная файловая система, создает абстрактный слой, скрывая детали оборудования, предоставляя общий файловый интерфейс для всех устройств
- Network Interface (NET) — сетевые интерфейсы, обеспечивает работу с различными сетевыми стандартами и сетевым оборудованием
- Inter-Process Communication (IPC) — межпроцессная подсистема, поддерживающая несколько механизмов для process-to-process связей в единой Linux-системе

Вспомогательные модули:

Утилиты

программы, решающие отдельные задачи управления и сопровождения компьютерной системы (дисковые утилиты: дефрагментаторы, очистка диска, разметка диска, программы сжатия, резервное копирование; утилиты работы с реестром; утилиты мониторинга оборудования и т.д.)

Системные обрабатывающие программы

текстовые и графические редакторы, компиляторы, компоновщики, отладчики

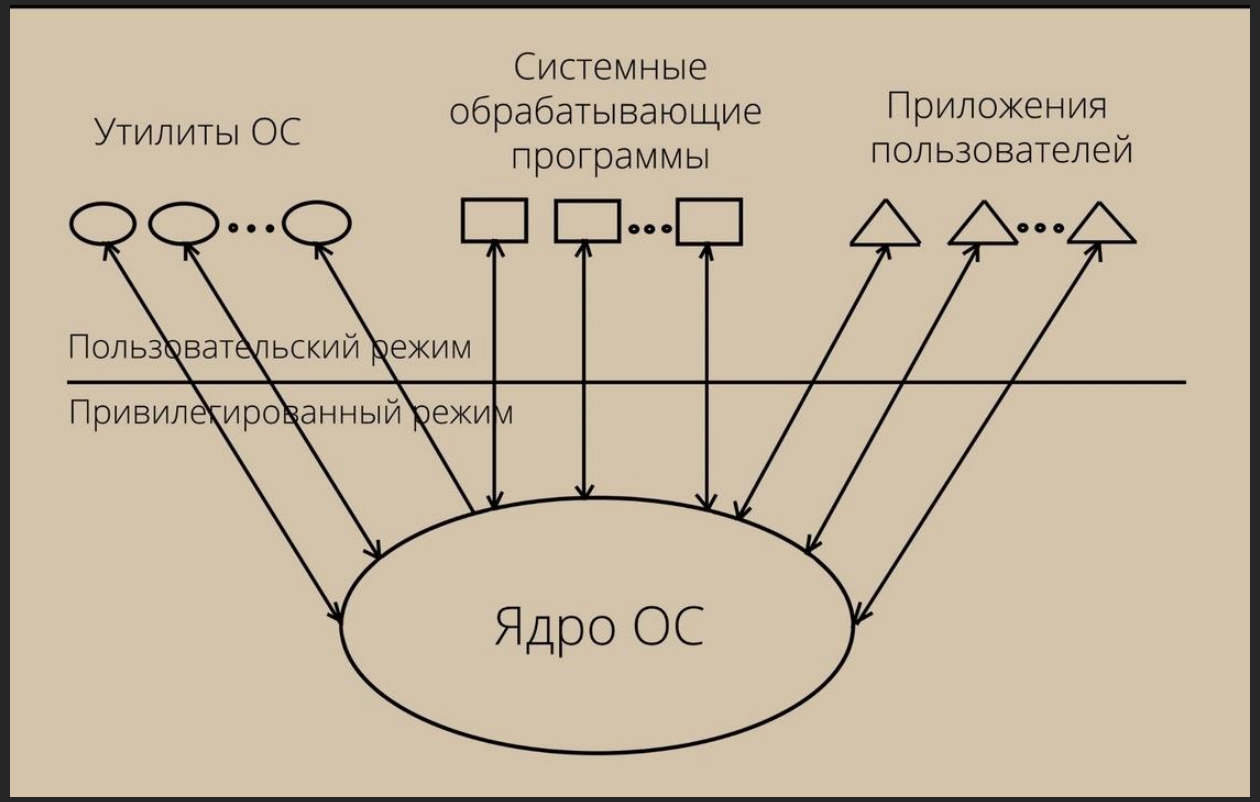
Программы дополнительных услуг

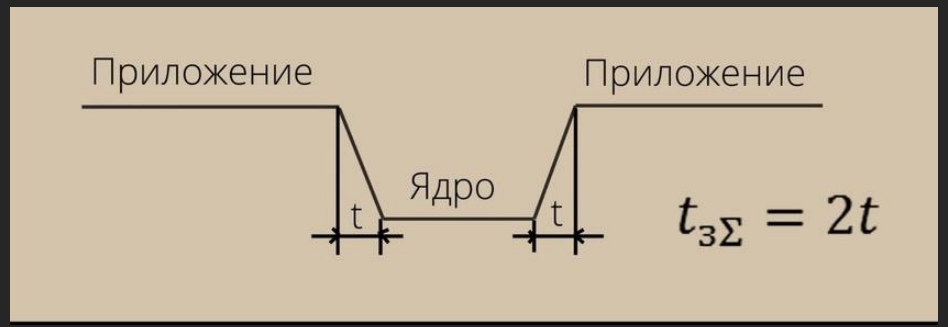
специальный вариант пользовательского интерфейса, калькулятор, игры

Библиотеки процедур

библиотеки математических функций, функций ввода/вывода и т.д.

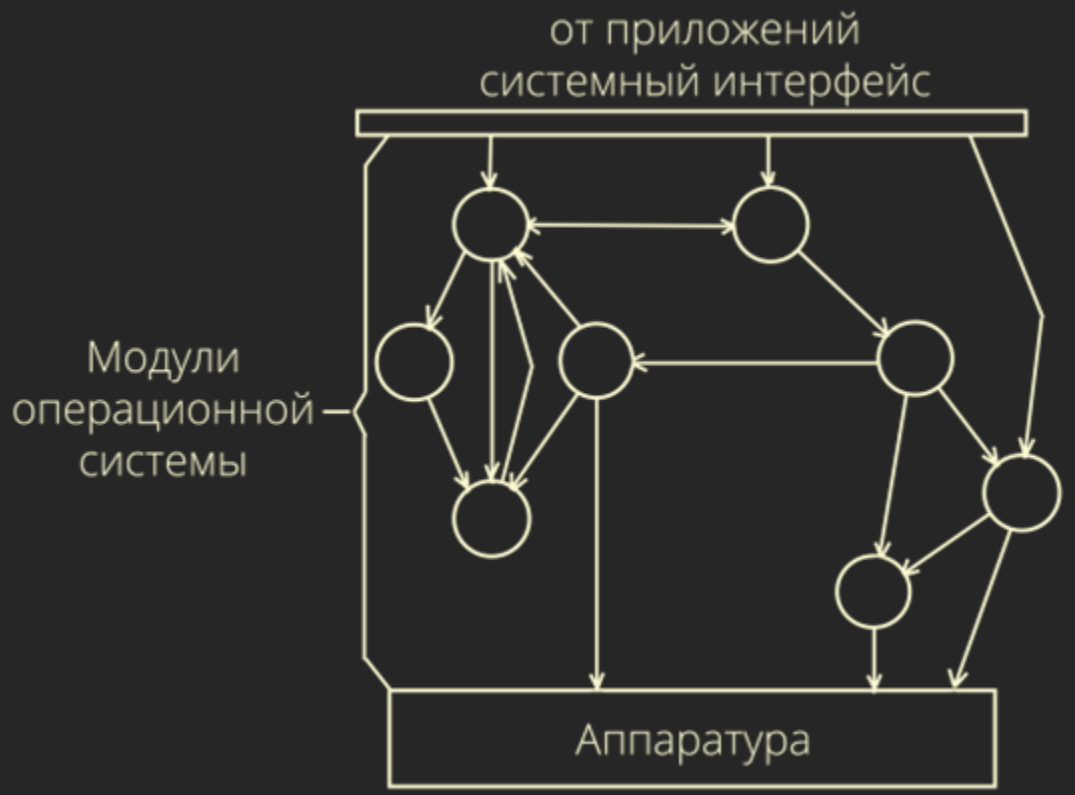
Классическая архитектура





Монолитная структура ОС

Вся ОС работает как единая программа в режиме ядра. Монолитная ОС написана как набор процедур, связанных в одну большую исполняемую программу. Каждая процедура имеет возможность при необходимости вызвать другую.

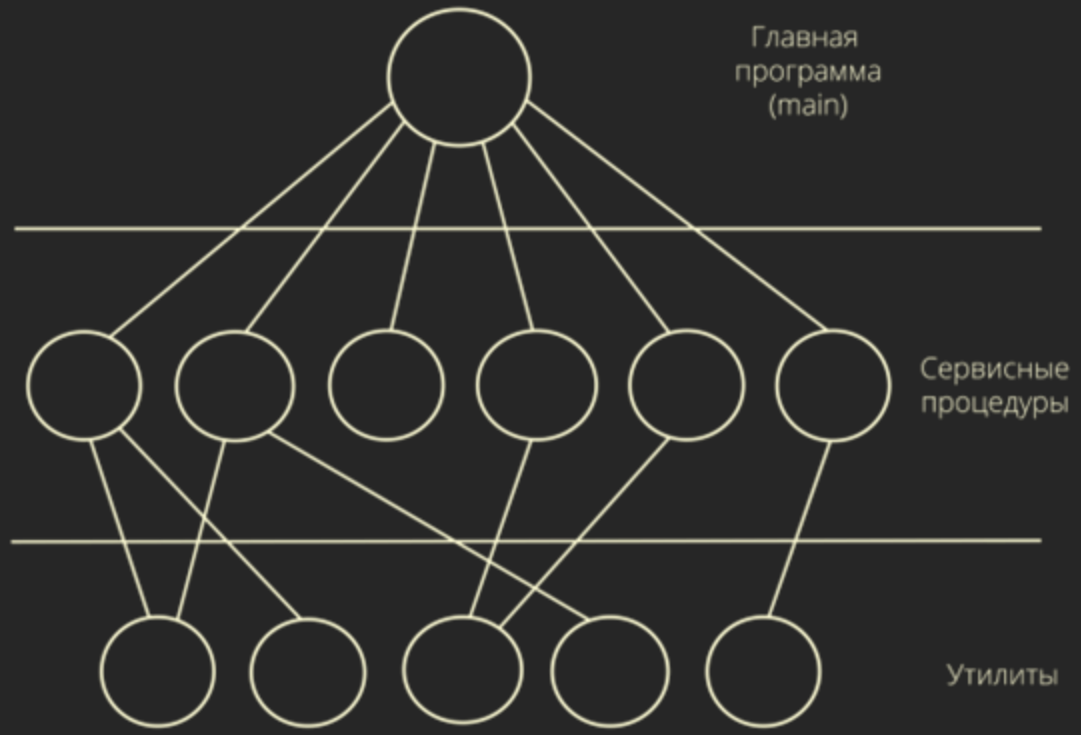


Монолитные системы могут быть структурированными:

Главная программа, которая вызывает требуемые сервисные процедуры.

Набор сервисных процедур, реализующих системные вызовы.

Набор утилит, обслуживающих сервисные процедуры.



Многоуровневые системы

Обобщением предыдущего подхода является организация ОС как иерархии уровней с хорошо определенными связями между ними, так чтобы объекты уровня N могли вызывать только объекты из уровня N-1.

Нижним уровнем в таких системах обычно является аппаратура, верхним уровнем интерфейс пользователя. Прикладные программы или модули самой операционной системы передают запросы вверх и вниз по этим уровням.

Плюсы:

- Многоуровневые системы хорошо реализуются.
- Слоеные системы хорошо модифицируются.

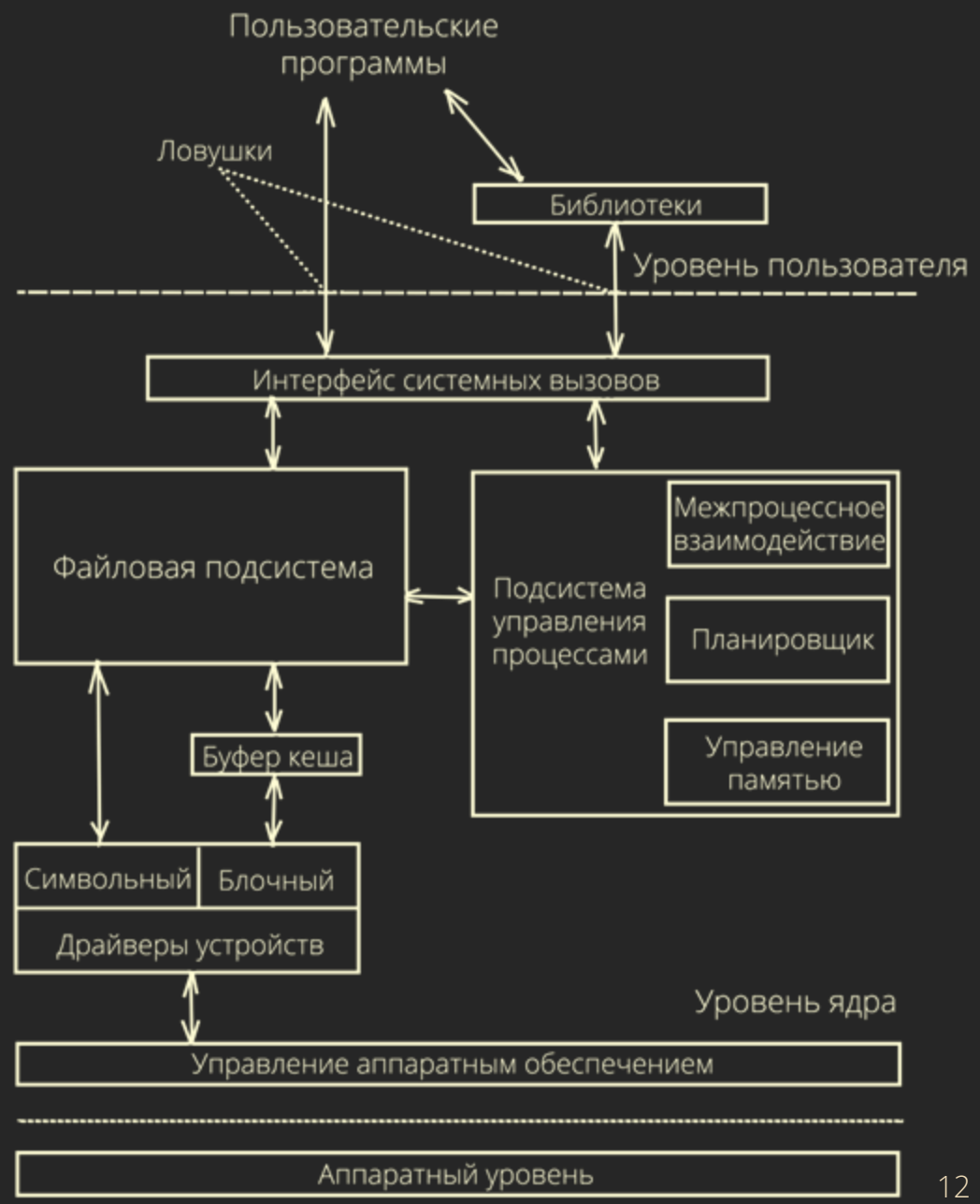
Минусы:

- Многоуровневые системы сложны для разработки.
- Менее эффективны, чем монолитные.

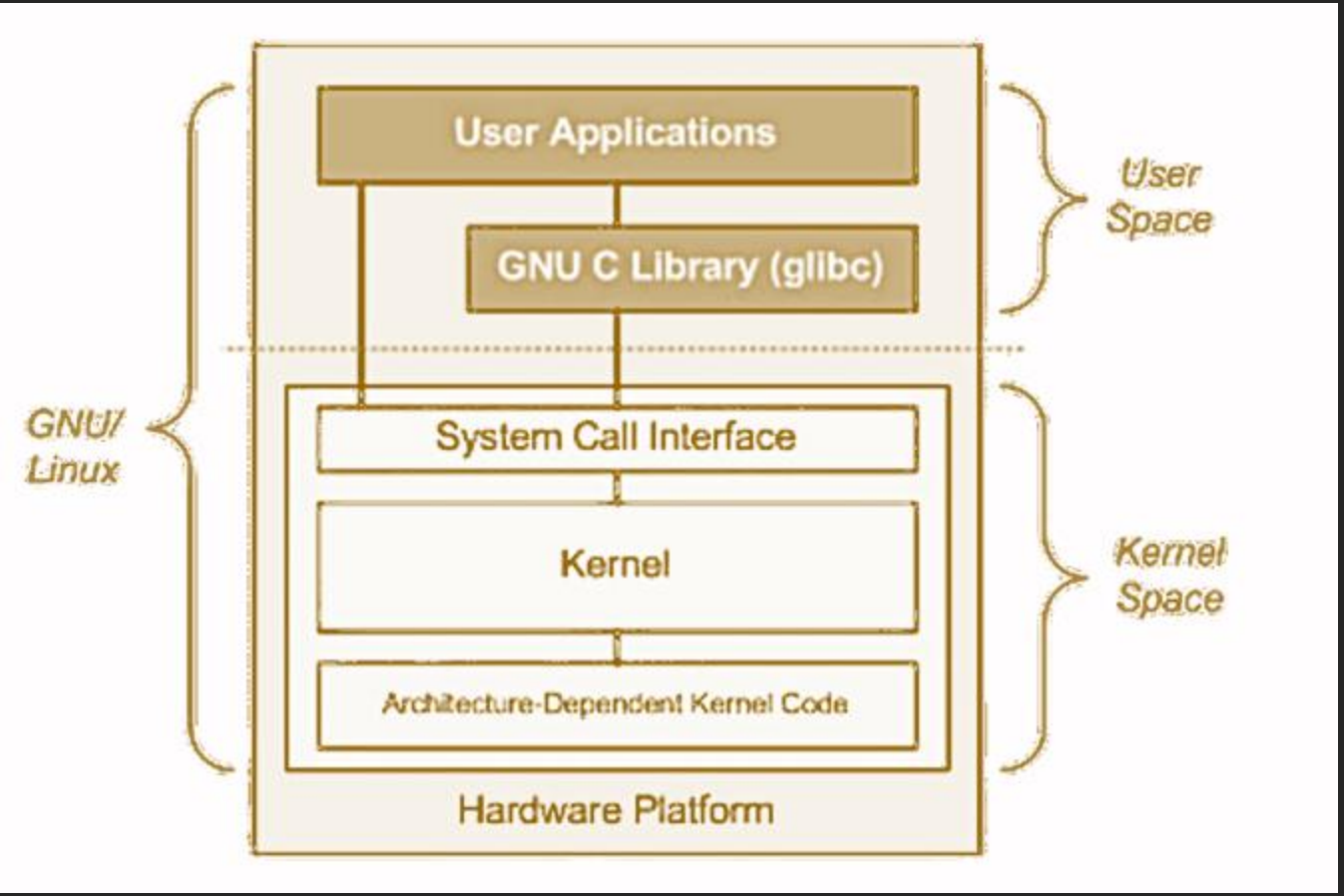
Традиционная архитектура UNIX



Традиционная архитектура UNIX

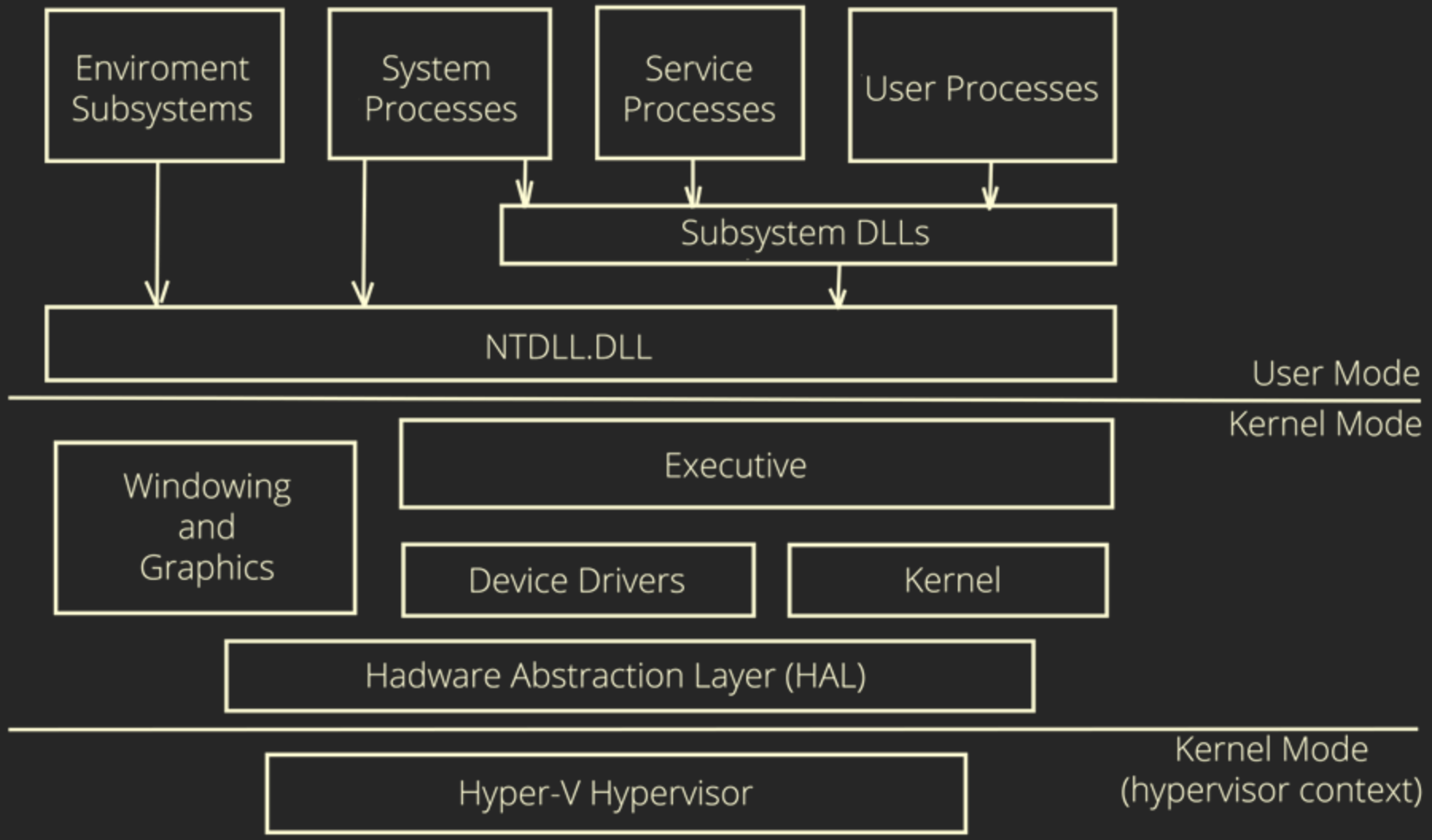


Linux

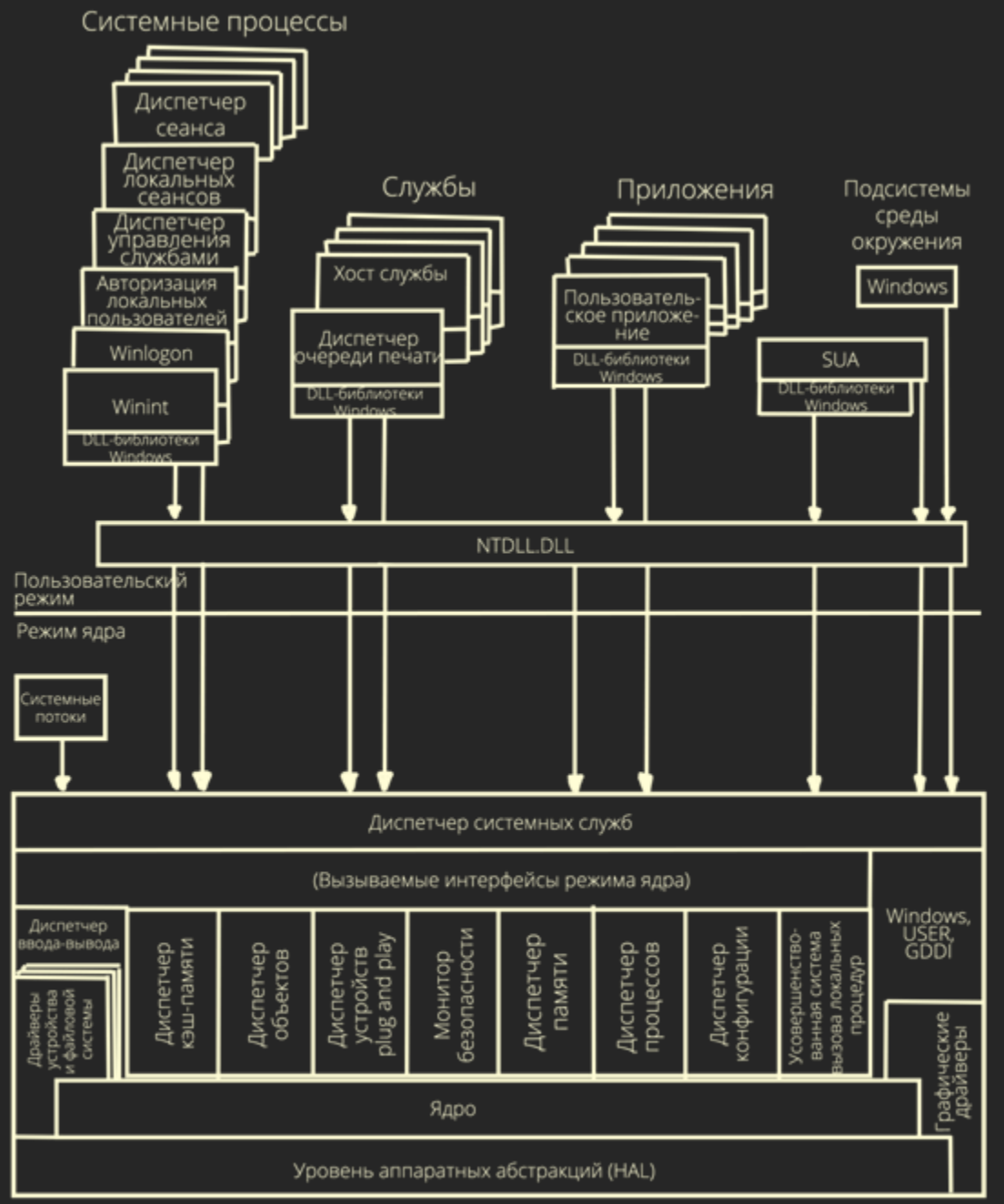

User Applications
GNU C Library (glibc)
System Call Interface
Kernel
Architecture-Dependent Kernel Code
Hardware Platform
User Space
Kernel Space
GNU/ Linux

Упрощенное представление архитектуры Windows



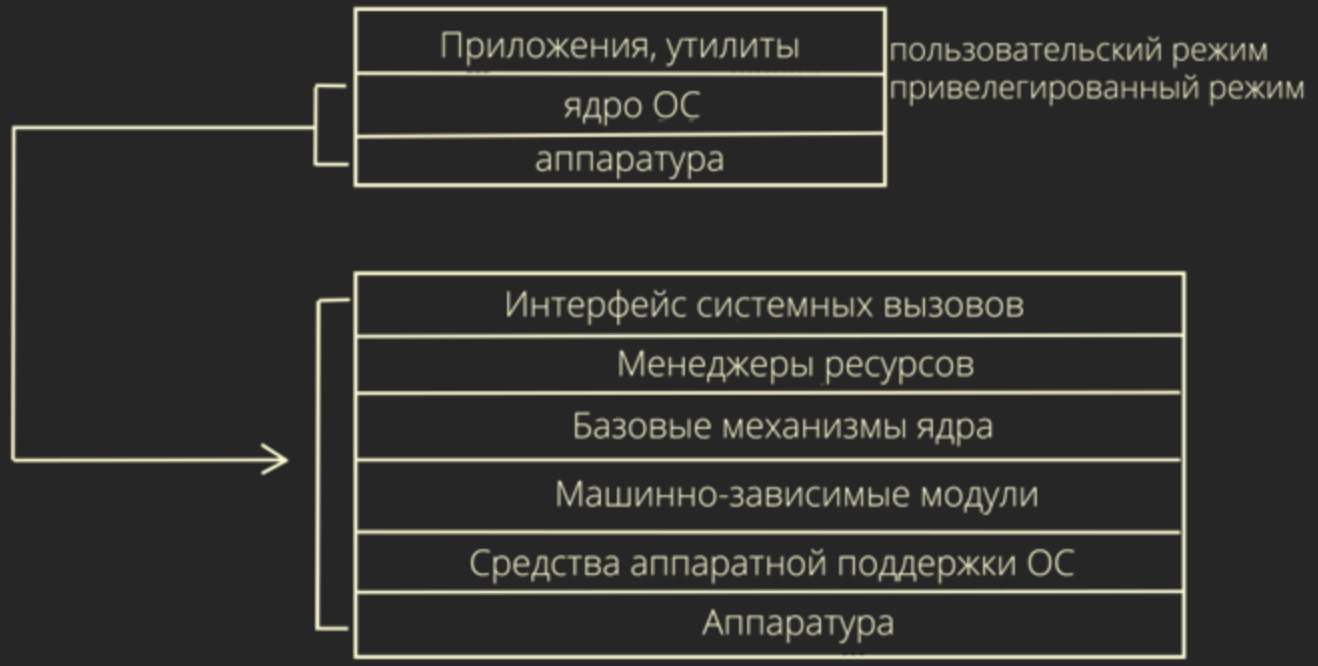

Enviroment Subsystems
System Processes
Service Processes
User Processes
Subsystem DLLs
NTDLL.DLL
User Mode
Kernel Mode
Windowing and Graphics
Executive
Device Drivers
Kernel
Hadware Abstraction Layer (HAL)
Kernel Mode (hypervisor context)
Hyper-V Hypervisor

Архитектура Windows



Аппаратные интерфейсы (шины, устройства ввода-вывода, прерывания, интервальные таймеры, DMA, управление кэш-памятью и т.д.)

Многослойная модель ядра



машинно-зависимые компоненты ОС

в идеале этот слой полностью экранирует вышележащие слои ядра от особенностей аппаратуры (пример – слой HAL ОС Windows NT)

базовые механизмы ядра

этот слой выполняет наиболее примитивные операции ядра, реализует решения о распределении ресурсов, принятые на более высоком уровне

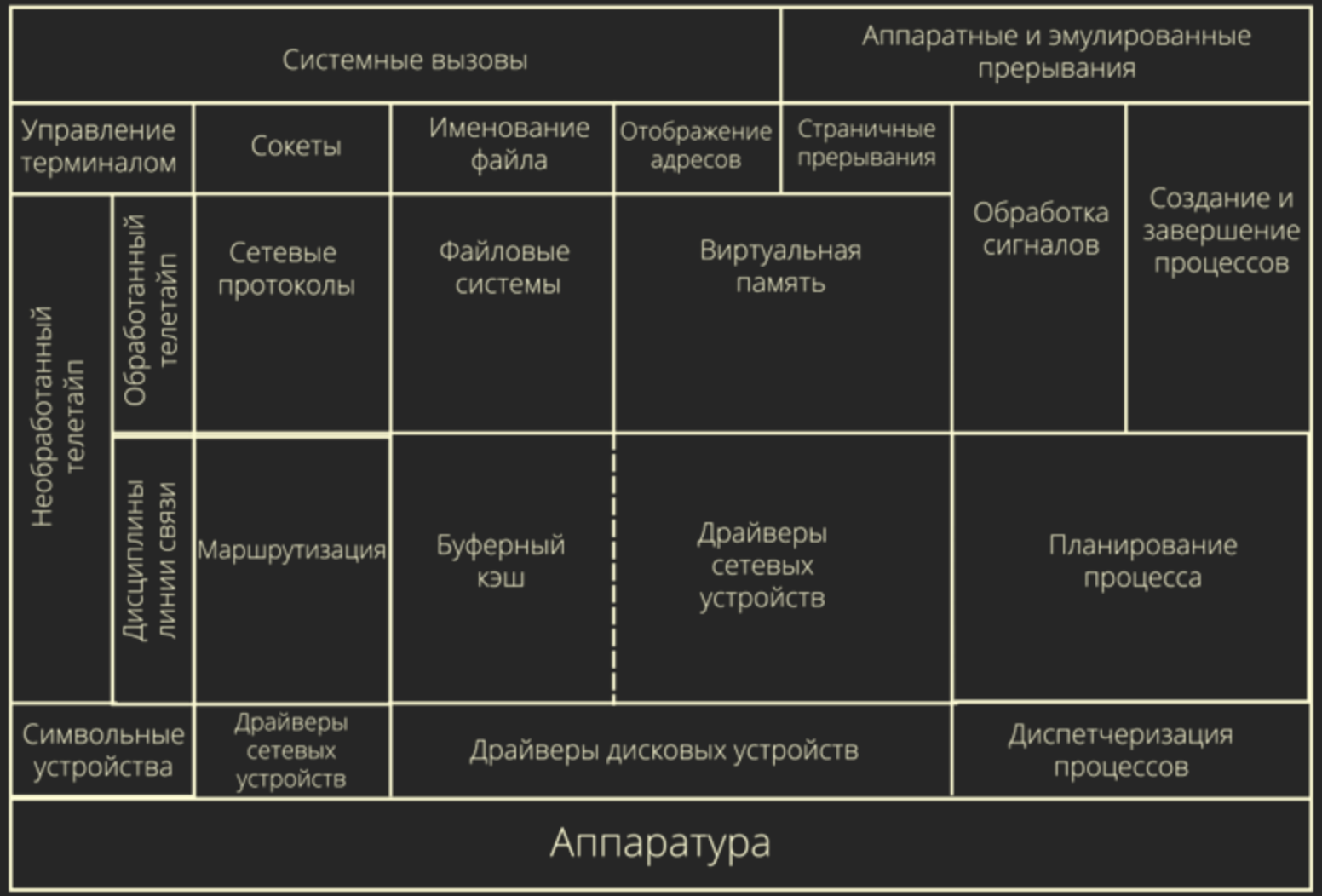
менеджеры ресурсов

слой состоит из мощных функциональных модулей, реализующих стратегические задачи по управлению основными ресурсами ОС

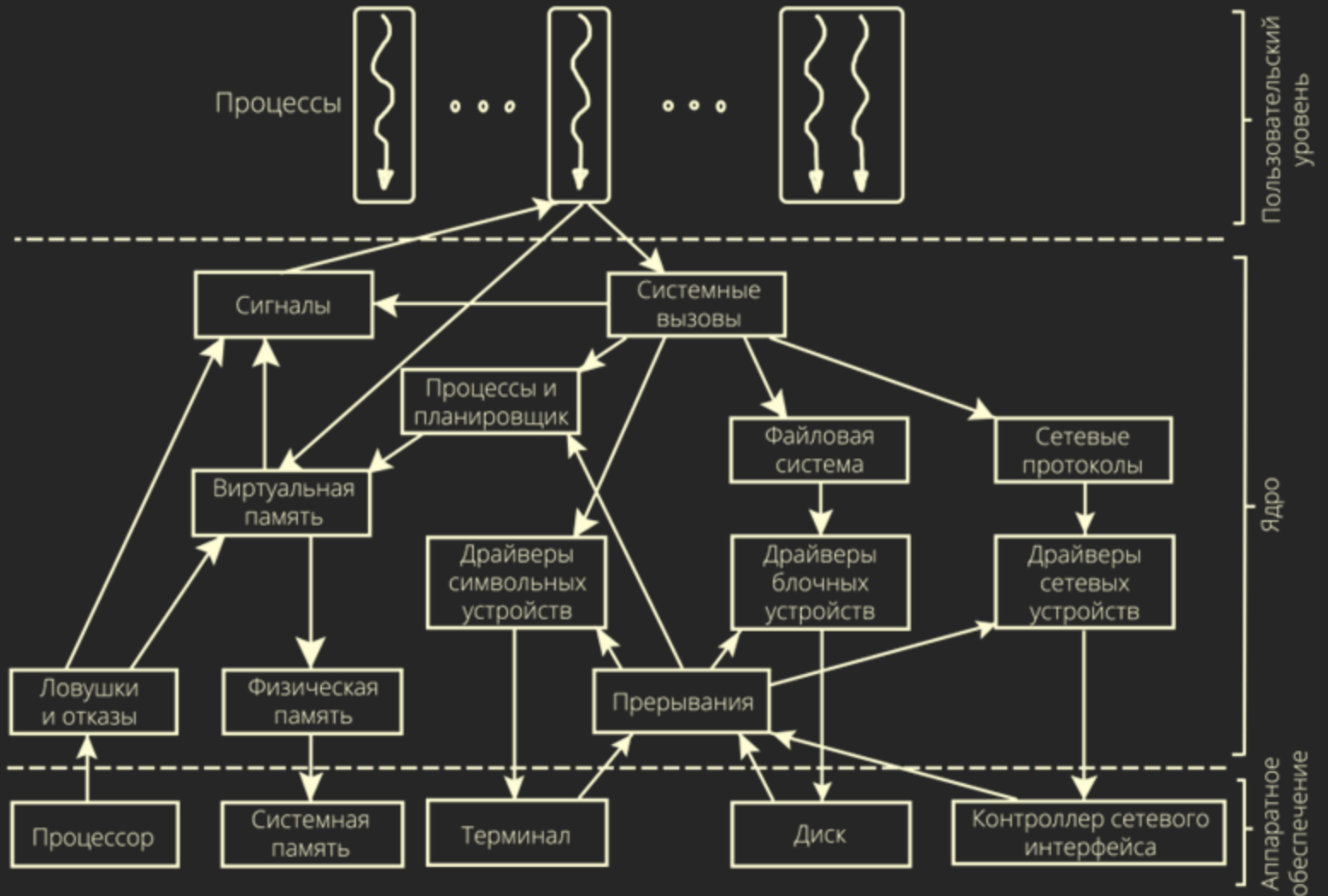
интерфейс системных вызовов

взаимодействует непосредственно с приложениями и системными утилитами, образуя прикладной программный интерфейс ОС

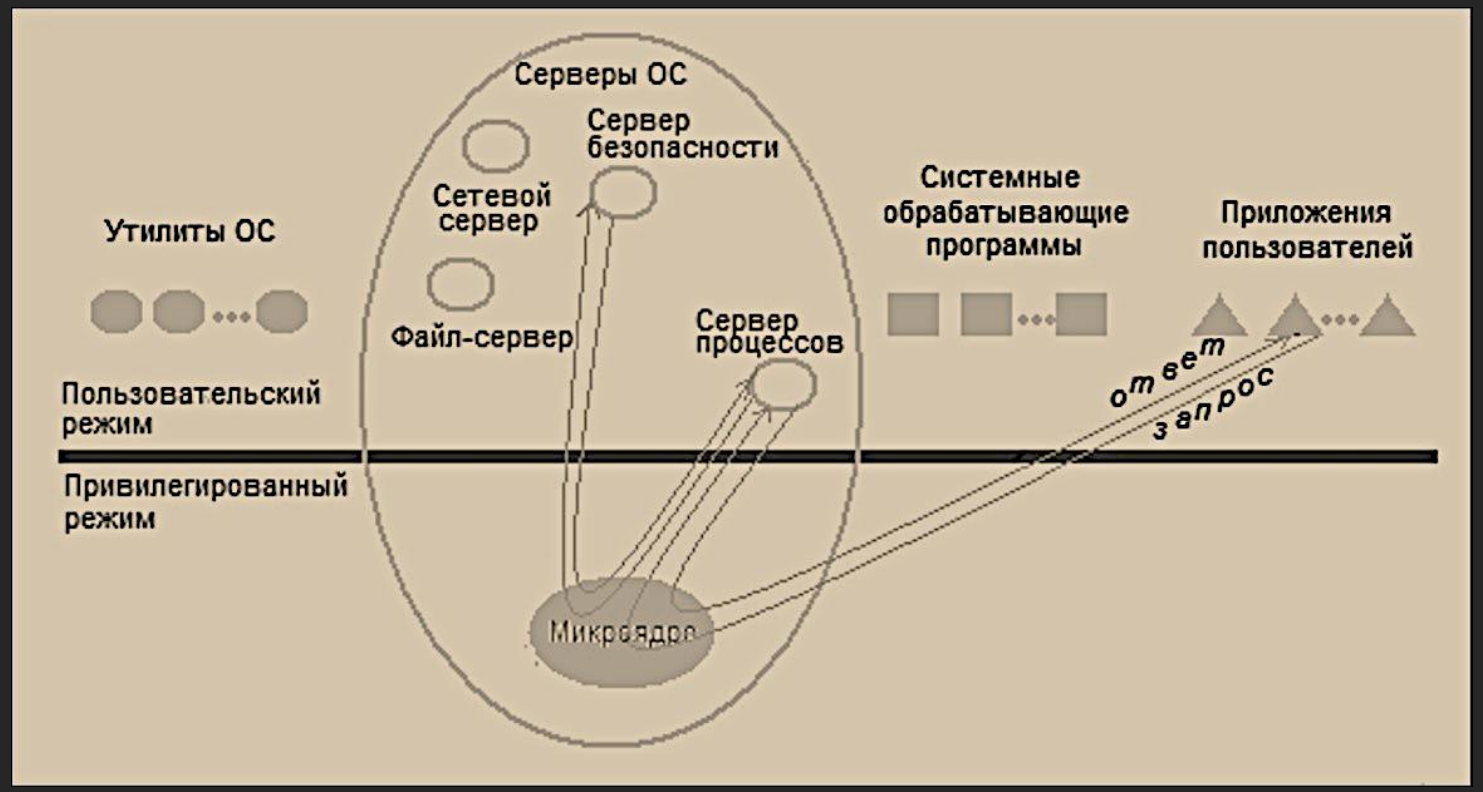
Ядро ОС UNIX

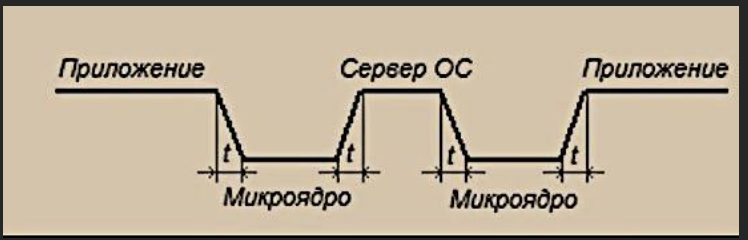

Системные вызовы
Аппаратные и эмулированные прерывания
Управление терминалом
Сокеты
Именование файла
Отображение адресов
Страничные прерывания
Обработка сигналов
Создание и завершение процессов
Необработанный телетайп
Обработанный телетайп
Сетевые протоколы
Файловые системы
Виртуальная память
Дисциплины линии связи
Маршрутизация
Буферный кэш
Драйверы сетевых устройств
Планирование процесса
Символьные устройства
Драйверы сетевых устройств
Драйверы дисковых устройств
Диспетчеризация процессов
Аппаратура

Компоненты ядра Linux



Микроядерная архитектура





$t_{з\Sigma}=4t$

Микроядерная архитектура

Достоинства

- **Повышенная безопасность**

Благодаря небольшому размеру ядра его код проще верифицировать, а само оно имеет меньшую поверхность атаки.

- **Масштабируемость**

В микроядерных ОС можно добавлять, удалять и обновлять системные сервисы, не затрагивая ядро. Это позволяет создавать индивидуальные конфигурации.

- **Отказоустойчивость**

Благодаря модульной архитектуре сбой в одном компоненте не повлияет на систему в целом. Изоляция компонентов микроядра не позволит потенциальным злоумышленникам получить полный доступ к критически важным сервисам.

Недостатки

- **Риск снижения производительности**

Так как отдельные компоненты ОС работают в разных пространствах, их взаимодействие может вызывать «накладные расходы» в межпроцессном взаимодействии. Это может привести к снижению производительности. Эта проблема была особенно актуальна, когда появлялись первые микроядерные операционные системы, но с ростом производительности аппаратных платформ ее влияние снижается. Сегодня существуют микроядерные ОС реального времени с высокими показателями быстродействия, например PikeOS.

- **Сложность проектирования системы**

Модульная структура создает дополнительные сложности при координации работы сервисов.

- **Сложности обеспечения аппаратной совместимости**

В микроядерных ОС различные компоненты системы работают независимо друг от друга. Это может вызвать проблемы в обеспечении поддержки драйверов и повлиять на их совместимость с аппаратным обеспечением.

Примеры микроядерных ОС

MINIX

Операционная система, изначально разработанная в качестве образовательного проекта, который помогал студентам изучать внутреннее устройство ОС и экспериментировать с ним. Отличается небольшим размером и упрощенной структурой. MINIX — это, возможно, самая распространенная микроядерная ОС в мире: она установлена в чипсетах Intel, а это две трети рынка процессоров с архитектурой x86.

QNX

Коммерческая микроядерная ОС, предназначенная для встраиваемых систем. QNX поддерживает многозадачность, обладает повышенной безопасностью и отказоустойчивостью, поэтому ее применяют в системах, где критически важны эти качества: в сферах автомобилестроения, здравоохранения, аэрокосмической промышленности и других.

L4

Семейство микроядер, разработанное для встраиваемых систем и мобильных устройств. Оно включает разные версии: Pistachio, Fiasco.OC, seL4. Микроядра L4 отличаются высокой производительностью, низкой задержкой, безопасностью и универсальностью. Благодаря этому сфера их применения достаточно широка.

Hurd

Операционная система, разработанная как альтернатива ядру Linux с упором на модульность и масштабируемость. Пользователи могут настраивать Hurd под свои потребности.

PikeOS

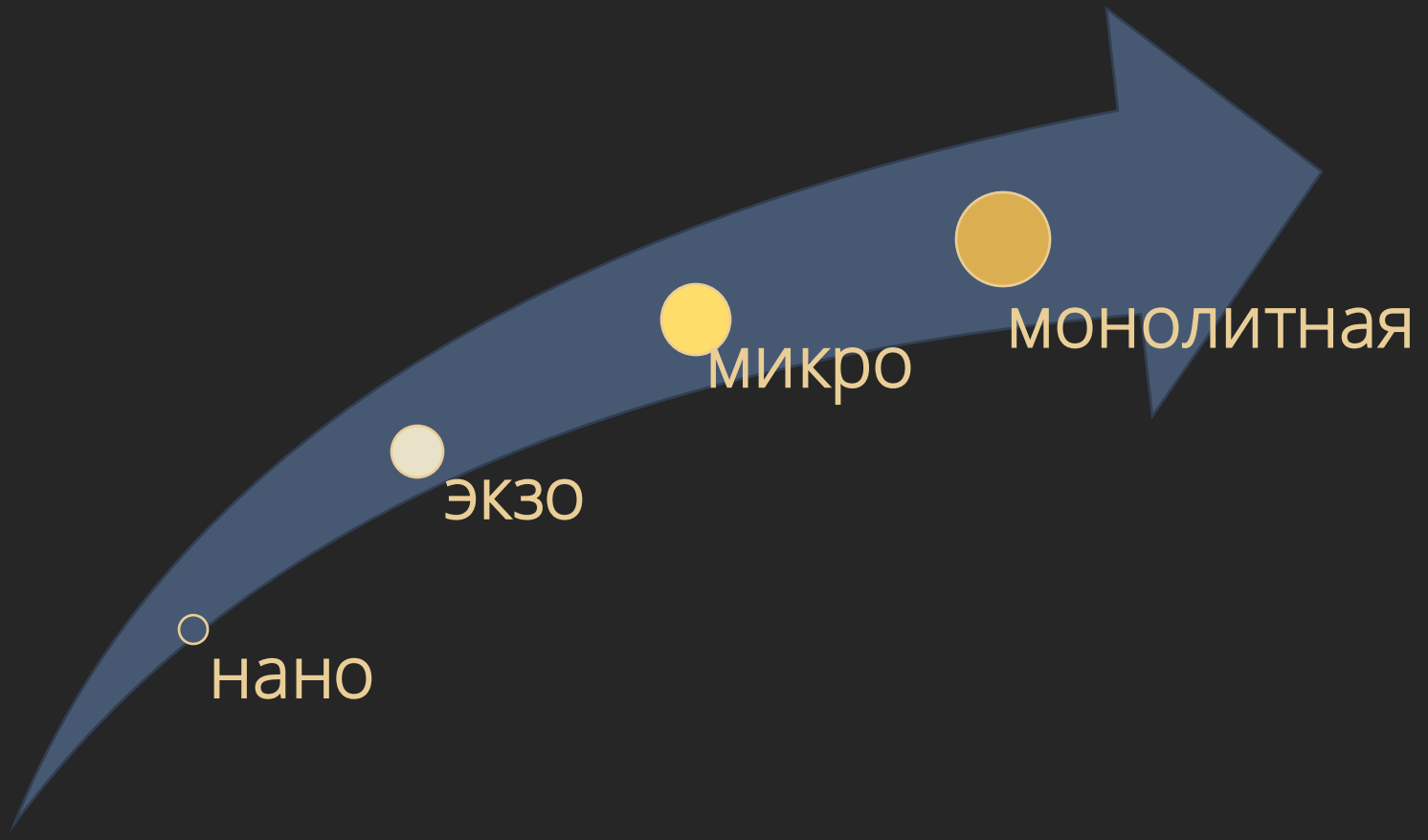
Микроядерная операционная система реального времени (RTOS), разработанная для встраиваемых систем с высокими требованиями к надежности и безопасности. Система применяется в авионике, автомобилестроении, интеллектуальных железнодорожных системах и других отраслях.

Экзоядро — ядро операционной системы компьютеров, предоставляющее лишь функции для взаимодействия между процессами и безопасного выделения и освобождения ресурсов.

Наноядро — архитектура ядра операционной системы компьютеров, в рамках которой упрощённое и минималистичное ядро выполняет лишь одну задачу — обработку аппаратных прерываний, генерируемых устройствами компьютера. После обработки прерываний от аппаратуры наноядро посылает информацию о результатах обработки вышележащему программному обеспечению при помощи того же механизма прерываний. Возможна минимальная поддержка потоков: создание и переключение.

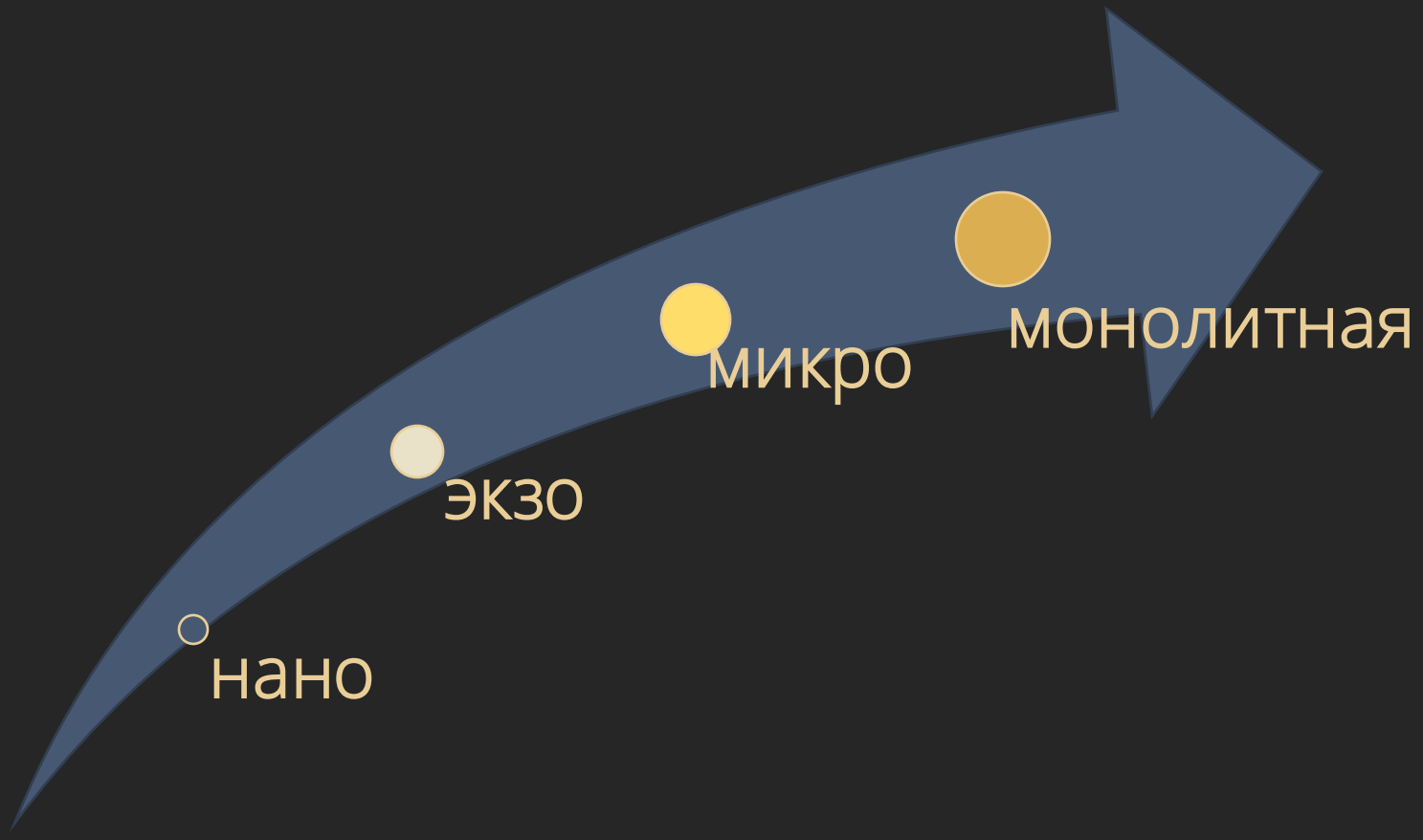
Сравнение моделей архитектур ОС

Производительность



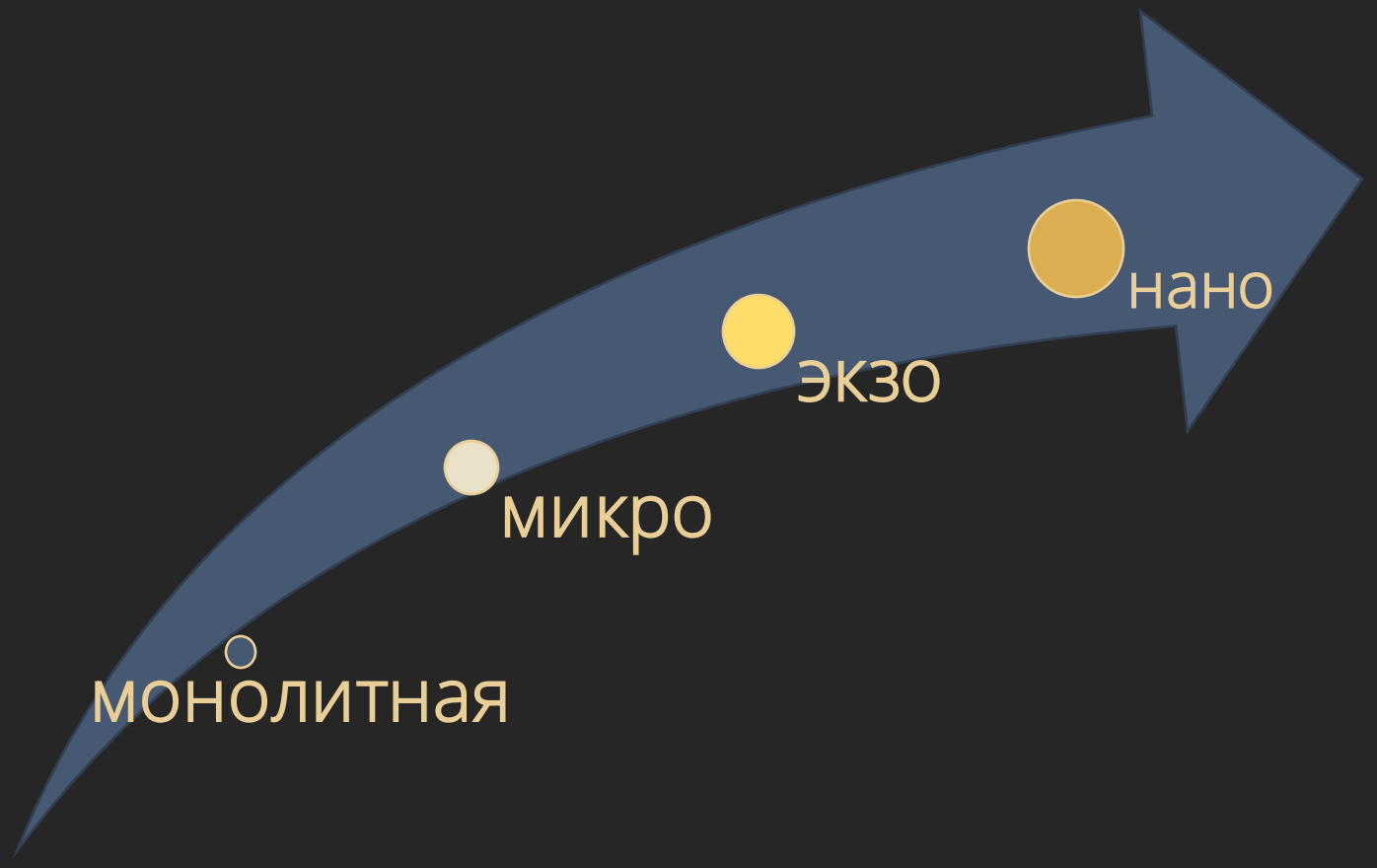
Сравнение моделей архитектур ОС

Объём привилегированного кода



Сравнение моделей архитектур ОС

Защищенность ядра



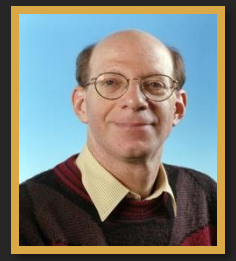
	Плюсы	Минусы
Микроядра	• При ошибке одного из серверов можно избежать падения всей системы • Запуск процессов в режиме ядра только тех серверов, которым это необходимо • Низкое потребление памяти	• Много накладных расходов на обеспечение взаимодействия между серверами • Процессы должны ждать свою очередь, чтобы получить информацию
Монолитные	• Прямой запуск функций • Производительность	• При любом изменении требуется пересборка всего ядра • Любая ошибка в любой подсистеме приводит к kernel panic

Спор Эндрю Таненбаума и Линуса Торвальдса

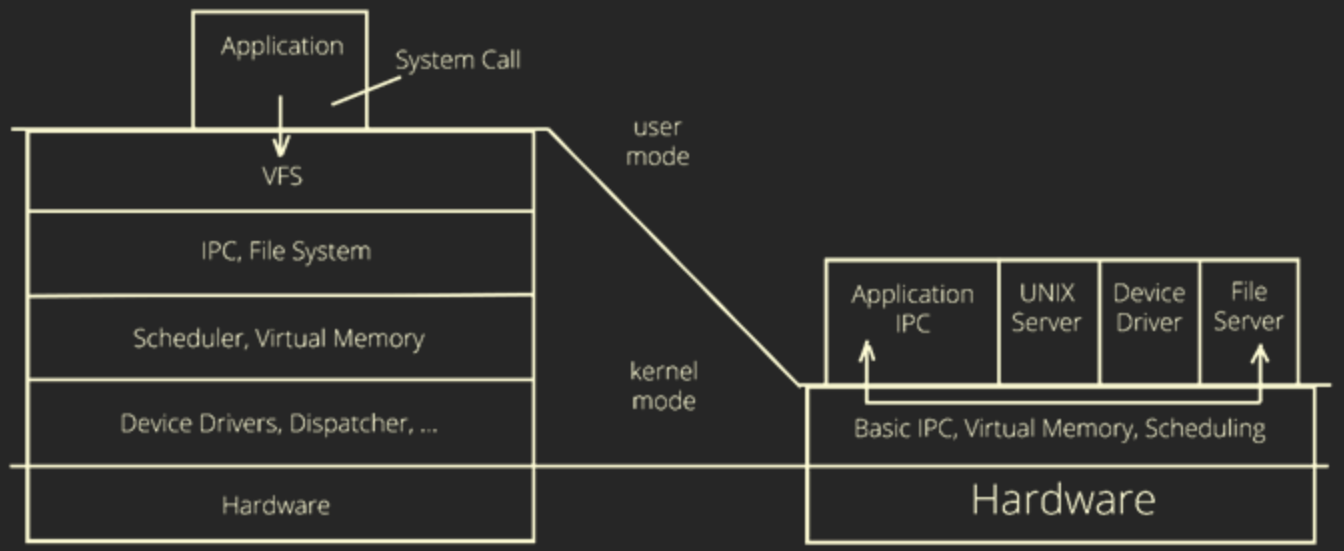
Linux
Монолитная архитектура

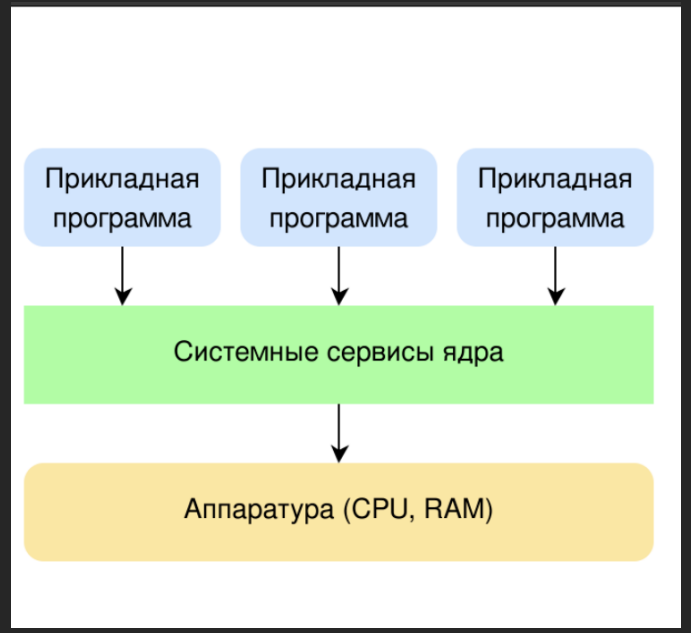


VS

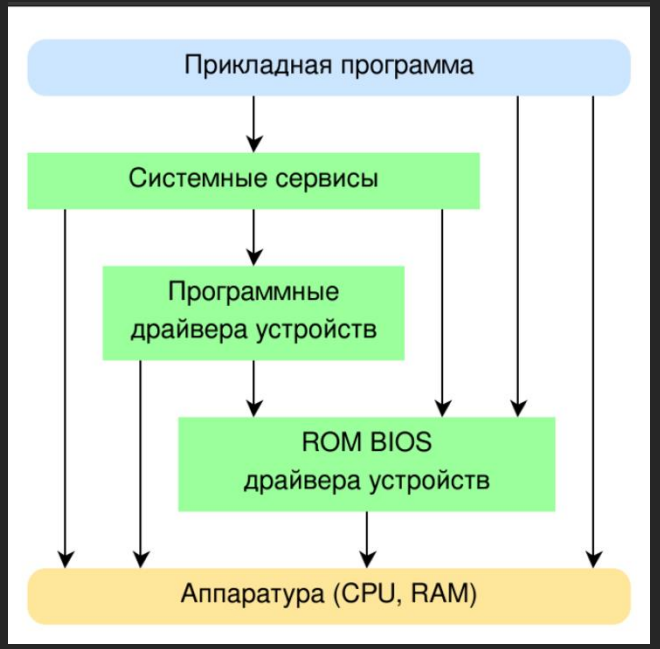


Minix
Микроядро

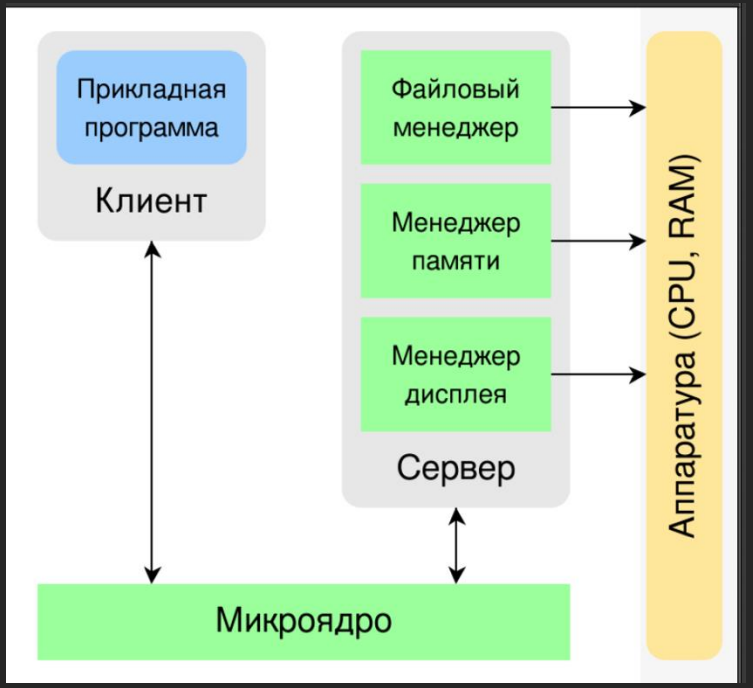




Монолитная архитектура



Уровневая (слоеная) архитектура



Архитектура «клиент-сервер»